Pontificia Universidad Católica Madre y Maestra

Asignatura:

Programación Web Avanzada

Profesor:

Carlos A. Camacho G.

Asignación:

Práctica 4 - Implementando sistemas basados en Bróker de Mensajería

Git:

https://github.com/firnex360/st-icc-354-001-ferb0001/tree/main/practica-4

Nombres & Matrículas:

María I. Tiburcio A.	*10150779*
Fernando E- Rodríguez B.	*10149894*

Fecha de Entrega:

9 de Marzo, 2026

República Dominicana, Santiago de los Caballeros

1. Uso de la Aplicación

Para correr el Docker Compose: dirigirse a la carpeta practica-4 desde la terminal y correr “docker compose up -d”. Este instalará todas las imágenes necesarias e iniciará el contenedor.

puertos:
Visual grafico: http://localhost:8080/
Panel de connection ActiveMQ: http://localhost:8161/admin/index.jsp
Para acceder al paneer usar user y password “admin”

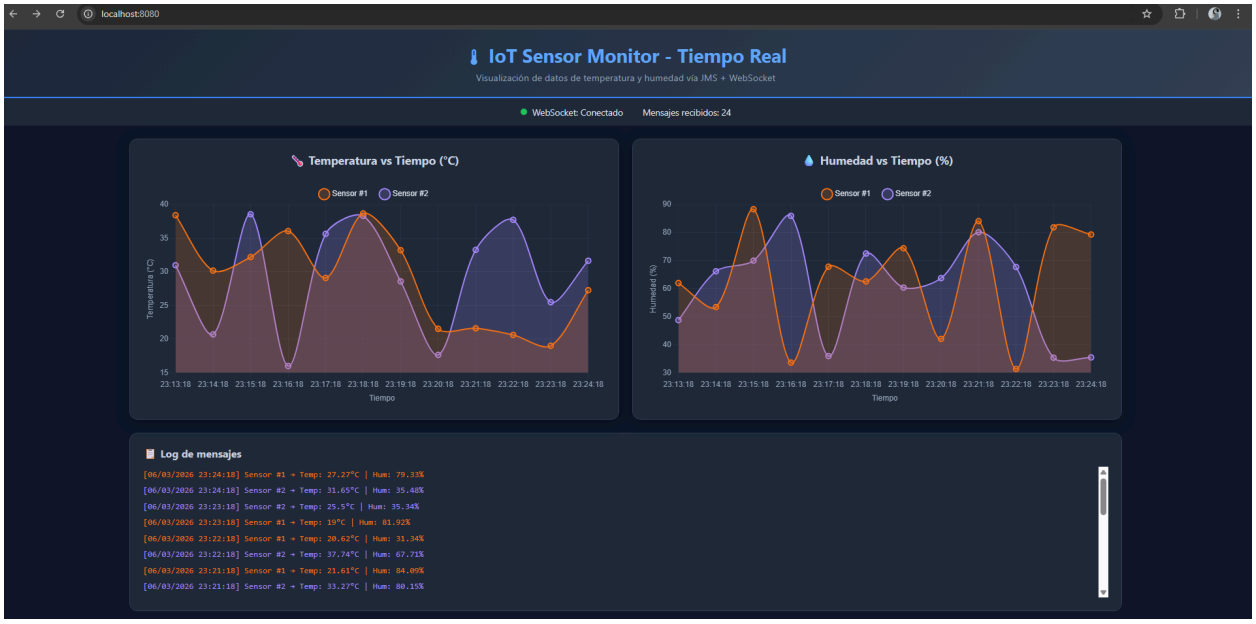
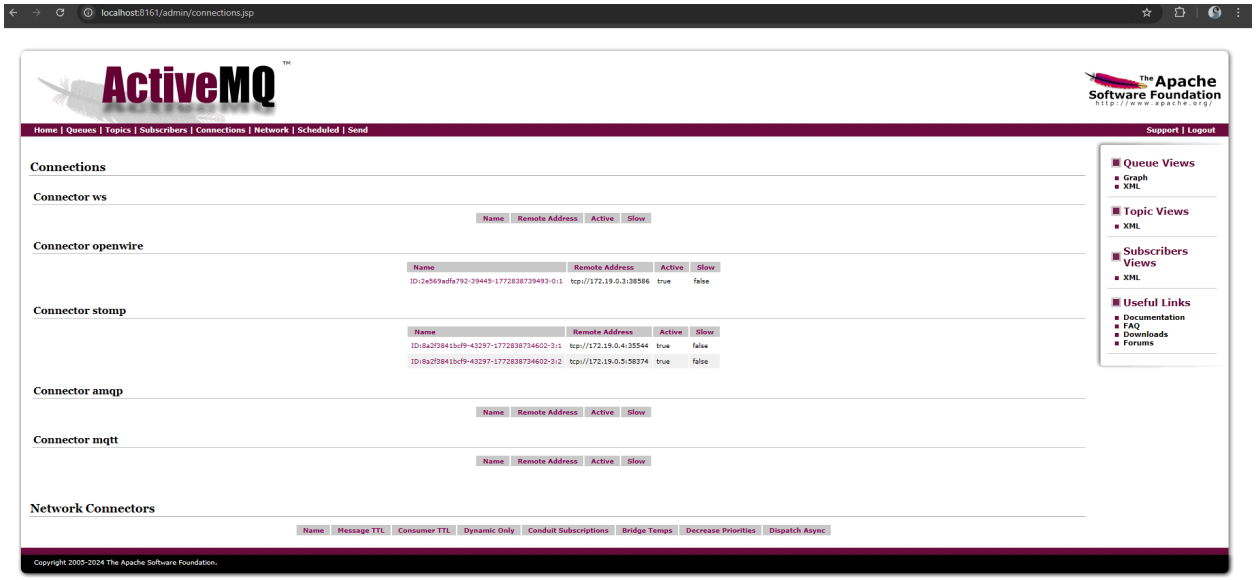
Introducción:
El proyecto consiste en el desarrollo de una arquitectura de mensajería orientada a Internet de las Cosas (IoT) utilizando el patrón de diseño Publish/Subscribe (Publicador/Suscriptor). El objetivo principal es recolectar datos ambientales (temperatura y humedad) generados por múltiples dispositivos simulados y visualizarlos en tiempo real a través de una interfaz web gráfica, garantizando escalabilidad y persistencia de información en una base de datos relacional. Todo el ecosistema ha sido aislado mediante contenedores Docker y orquestado de manera integral empleando Docker Compose, logrando una plataforma portable, escalable y con capacidad de despliegue automatizado.

Desarrollo:
La implementación del proyecto se ha dividido en cuatro componentes principales que interactúan entre sí dentro de la misma red de contenedores:

1. Broker de Mensajería (Apache ActiveMQ): Actúa como el núcleo de comunicación asíncrona. Se configuró para habilitar tanto conexiones OpenWire (JMS) como STOMP, utilizando un tópico específico llamado `notificación sensores` al que los clientes pueden publicar y suscribirse.

2. Clientes IoT Simulados (Python): Se desarrollaron scripts en Python para emular los 'Endpoints' o sensores. Estos clientes se conectan utilizando el protocolo ligero STOMP, generando intervalos regulares de lecturas aleatorias estructuradas en formato JSON (incluyendo fecha, ID del dispositivo, temperatura y humedad) que son transmitidas constantemente al broker.

3. Backend Server (Java / Spring Boot): El servidor principal ha sido construido usando Spring Boot 4.0.3 e implementando el estándar Java Message Service (JMS). Este servidor se subscribe continuamente al tópico de ActiveMQ. Tras la llegada de un mensaje (en formato de bytes UTF-8), el backend lo deserializar, lo almacena de manera persistente en una base de datos relacional incrustada (H2) y finalmente enruta los paquetes hacia los clientes web en tiempo real gracias a su integración de soporte WebSocket (con protocolo STOMP).

4. Frontend / Interfaz Gráfica (HTML, JS y Chart.js): Se desarrolló una aplicación web con una vista de control moderno y dinámico ("Dashboard"). Este se conecta mediante una

conexión WebSocket remota asíncrona hacia el Backend Spring Boot y se nutre en vivo de los datos recién ingeridos. Usa la librería gráfica `Chart.js` para renderizar visualmente y sin recargas dos gráficos lineales temporales que separan la "Temperatura" de la "Humedad" mapeando a cada cliente (Sensor #1 y Sensor #2) de una forma visualmente diferenciada mediante colores.